

gRPC Integration with Talend Studio

Proof of Concept — Summary Document

April 2026 | Data Engineering Team

Objective

Build a Proof of Concept to demonstrate how Talend Studio can consume data from a gRPC-based microservice, using a locally built Java gRPC service as the test target before integrating with the actual microservice team's service.

Phase 1: Environment Setup

- Installed and configured Apache Maven 3.9.15 on Windows
- Verified Java JDK 17.0.15 as the runtime
- Created a Maven project (employee-grpc-service) using the maven-archetype-quickstart archetype

Phase 2: Project Configuration

Configured pom.xml with the following key dependencies and plugins:

Component	Purpose
grpc-netty-shaded	HTTP/2 transport layer for gRPC
grpc-protobuf	Protobuf message integration
grpc-stub	Auto-generated client/server stub support
protobuf-java	Protobuf Java runtime
protobuf-maven-plugin	Compiles .proto files and auto-generates Java stubs during build
maven-shade-plugin	Packages everything into a single runnable fat JAR

Phase 3: Service Contract Definition (.proto)

Defined the gRPC service contract in employee.proto with one data model and two RPC methods:

Data Model — Employee fields: id, name, department, email, salary

Method	Type	Description
GetEmployee	Unary	Takes an employee ID, returns a single employee record
ListEmployees	Server Streaming	Takes a department name, streams back all matching employees

Java stubs were auto-generated from the .proto file using mvn generate-sources, producing ready-to-use classes under target/generated-sources/protobuf/.

Phase 4: Service Implementation

Implemented EmployeeServiceImpl.java extending the auto-generated EmployeeServiceGrpc.EmployeeServiceImplBase:

- Loaded 8 dummy employee records across 4 departments: Engineering, HR, Finance, Marketing
- GetEmployee — looks up employee by ID, returns NOT_FOUND error if missing
- ListEmployees — filters by department and streams each record individually, returns NOT_FOUND error if department has no employees
- Used StreamObserver pattern (onNext, onCompleted, onError) for all responses

Phase 5: Server Setup

Implemented EmployeeServer.java:

- gRPC server starts on port 50051 (standard gRPC port)
- Registers EmployeeServiceImpl as the service handler
- Includes graceful shutdown via JVM shutdown hook (handles Ctrl+C)
- Packaged as a fat JAR using Maven Shade plugin and launched via:
`java -jar target\employee-grpc-service-1.0-SNAPSHOT.jar`

Phase 6: Testing with Kreya

Validated the running gRPC server using Kreya (GUI gRPC client tool, similar to Postman):

- Imported employee.proto into Kreya to auto-discover service methods
- Connected to localhost:50051 with TLS disabled
- Successfully tested all four scenarios:

Test	Method	Input	Result
Valid employee lookup	GetEmployee	E003	Returned Carol White's record 
Invalid employee lookup	GetEmployee	E999	Returned NOT_FOUND error 
Valid department stream	ListEmployees	Engineering	Streamed 3 employees 
Invalid department stream	ListEmployees	Legal	Returned NOT_FOUND error 

Current Status

-  Environment setup (Maven 3.9.15 + Java 17)
-  Maven project structure created
-  pom.xml configured with gRPC dependencies
-  employee.proto service contract defined
-  Java stubs auto-generated from .proto
-  EmployeeServiceImpl implemented with dummy data
-  gRPC server running on port 50051
-  Validated with Kreyo GUI client
- ☐ Integrate with Talend Studio using tJavaFlex

Next Steps

- Export the generated stub JARs and gRPC dependency JARs
- Add JARs as external modules in Talend Studio
- Build a Talend job using tJavaFlex to call GetEmployee and ListEmployees
- Validate data flow from gRPC service into Talend pipeline
- Once validated locally, point Talend at the actual microservice team's gRPC endpoint